

Universitatea Tehnică „Gheorghe Asachi” din Iași
Facultatea de Automatică și Calculatoare
Domeniul: Ingineria sistemelor
Specializarea: Automatică și Informatică Aplicată
Anul universitar 2025-2026

Proiect la disciplina
Comunicații în sisteme de conducere
Tema 4 - MS

Nume și prenume student:	Grupa:	Rol (C,T,R,U)
Carează Anamaria	1401A	C
Rotariu Lavinia-Monica	1401A	T
Badale Alexandru-Marian	1401A	R
Scutaru Maria-Cristina	1401A	U

Cuprins

- 1. Tema proiectului**
 - 2. Scurtă prezentare a resurselor HW și SW utilizate**
 - 3. Descrierea protocolului**
 - 4. Interfața I/E și conectarea la mediul de comunicare**
 - 5. Programul principal**
 - 6. Interfața cu utilizatorul**
 - 7. Transmisia mesajelor**
 - 8. Recepția mesajelor**
 - 9. Rezultate și concluzii**
- Bibliografie**

1. Tema proiectului

Proiectul presupune implementarea unui protocol de comunicație serială pentru o rețea de 3 noduri interconectate, realizată pe microsistemul BIG8051 echipat cu microcontrolerul C8051F040 de la Silicon Labs. Comunicația se face pe magistrala RS-485, prin portul serial UART1 în modul multiprocesor cu 9 biți.

Parametru	Valoare / Descriere
Port serial	UART1
Mediu fizic	RS-485 cu cablu, prin adaptor extern TTL-RS-485
Parametri comunicație	115200 baud, 9 biți, fără paritate, 1 bit de stop
Mod UART	Multiprocesor: primul octet (adresa HW dest) are bitul 9 = 1; toți ceilalți octeți au bitul 9 = 0
Codificare mesaj	Binar - câmpurile sunt transmise direct ca octeți binari
Protocol acces la mediu	Master-Slave (MS): un nod devine master după detectarea unui timeout pe magistrală
Număr noduri	3 noduri (adrese 0, 1, 2); ADR_NOD se setează la compilare în Protocol.h
Deteție erori	LRC (Longitudinal Redundancy Check) - XOR acumulat al tuturor octeților din mesaj
Lungime maximă date	10 octeți (NR_CHAR_MAX = 10)
Timeout slave	2×WAIT + ADR_NOD×WAIT ms (WAIT = 200 ms)

Mesajele transmise pe magistrala RS-485 au două formate distincte, în funcție de tipul mesajului:

Mesaj POLL_MES (tipmes = 0) – interogare sau răspuns implicit:

Adr HW destinație (1 octet, bit9=1)	Adr HW sursă (1 octet, bit9=0)	Sumă de control LRC (1 octet)
adresa_hw_dest	adresa_hw_src	sc = adresa_hw_dest XOR adresa_hw_src

Mesaj USER_MES (tipmes = 1) – mesaj de date:

Adr HW dest	Adr HW src	Tip (=1)	Src inițial	Dest final	Lungime	Date (max 10 B)	SC LRC
1 B (bit9=1)	1 B	1 B	1 B	1 B	1 B	Ing octeți	1 B

Aplicația implementează o rețea de 3 noduri echivalente care comunică pe o magistrală RS-485 half-duplex partajată. Fiecare nod rulează același cod binar, compilat cu constanta ADR_NOD specifică fiecăruia (0, 1 sau 2), definită în Protocol.h.

La pornire, toate nodurile încep ca slave și ascultă magistrala. Nodul care nu recepționează niciun mesaj în intervalul $2 \times \text{WAIT} + \text{ADR_NOD} \times \text{WAIT}$ ms (eșalonat per nod pentru a evita coliziunile la pornire simultană) se autoproclamă master. Masterul interoghează ciclic celelalte noduri, transmitând fie date pregătite de utilizator, fie mesaje POLL implicite. Slave-ul răspunde cu datele sale sau cu un POLL implicit. Mesajele slave-to-slave sunt rutate prin master.

Detecția coliziunilor pe RS-485 se realizează în TxMesaj(): după transmiterea octetului de adresă, nodul citește imediat magistrala; dacă octetul recepționat diferă de cel transmis, transmisia este anulată și se afișează mesajul de eroare „Detectie coliziune!”.

Suma de control (SC) LRC este calculată prin XOR în lanț al tuturor octeților din mesaj, excluzând octetul SC însuși. Această metodă detectează erorile de bit singular și este eficientă computațional pe arhitecturi 8051.

2. Scurtă prezentare a resurselor HW și SW utilizate

Hardware: Microsistemul BIG8051 (MikroElektronika) cu microcontrolerul C8051F040 (Silicon Laboratories), un adaptor extern TTL–RS-485 (tip MAX485) conectat la UART1, un afișaj LCD alfanumeric 2×16 și o tastatură matricială pentru interfața cu utilizatorul, și modulul USB-UART (FT232RL) pentru comunicația cu terminalul PC prin UART0.

Software: Mediul de dezvoltare Keil PK51 (μVision), cu compilatorul C51 și debugger JTAG prin Silicon Labs USB Debug Adapter. Codul este organizat în fișierele principale MS-JT.c, TxMesajV4.c, RxMesajV4.c, UserIO.c, un set de drivere de periferice reutilizabile (uart1.c, lcd.c, keyb.c etc.) și fișierul Protocol.h cu toate constantele și structurile de date ale aplicației.

3. Descrierea protocolului

3.1. Protocolul Master-Slave (MS)

Rețeaua folosește un protocol Master-Slave dinamic: la pornire toate nodurile ascultă magistrala ca slave. Primul nod care nu recepționează nimic în intervalul $2 \times \text{WAIT} + \text{ADR_NOD} \times \text{WAIT}$ ms ($\text{WAIT} = 200$ ms) preia rolul de master — timeout-ul eșalonat per adresă evită coliziunile la pornire simultană.

3.2. Fluxul de comunicație

Masterul interogă ciclic fiecare slave: trimite POLL_MES dacă nu are date, sau USER_MES dacă are text pregătit pentru acel nod. Slave-ul răspunde simetric. Mesajele slave-to-slave sunt rutate prin master, care le stochează și le retransmite la interogarea slave-ului destinatar.

3.3. Nivelul fizic RS-485

UART1 este configurat în mod 9 biți (modul multiprocesor) — primul octet al fiecărui mesaj are bitul 9 = 1 (adresă), restul au bitul 9 = 0 (date), astfel slave-urile neavizate ignoră mesajul în hardware. Detecția coliziunilor se face prin compararea eco-ului de pe magistrală cu octetul transmis; la discrepanță, transmisia se anulează și se aplică un delay de WAIT ms.

3.4. Organizarea buffer-elor

Fiecare nod menține tabloul `reteza[3]`, unde `reteza[i]` conține structura binară bufbin (câmpurile mesajului), flagul full (1 = mesaj pregătit, 0 = liber) și un buffer ASCII nefolosit în Tema 4. Flagul full este setat de UserIO sau RxMesaj și resetat de TxMesaj după transmisie.

4. Interfața I/E și conectarea la mediul de comunicație (nivelul fizic)

4.1. Configurarea porturilor

Crossbar switch-ul (XBR0–XBR3) alocă perifericele la pini: UART0 pe P0.0/P0.1 (USB-UART, 115200 baud, 8 biți), UART1 pe P0.6/P0.7 (RS-485, 115200 baud, 9 biți), P1.2/P1.3 pentru controlul transceiverului RS-485 (DE/\RE), Port 3 pentru LCD, Port 0 parțial pentru tastatură.

4.2. UART1 și modul multiprocesor

UART1 funcționează în mod 9 biți la 115200 baud (Timer 1, TH1 = 0x93, SYSClk = 25 MHz). Comutarea între MULTIPROC_ADRESA (bitul 9 = 1 — toți receptorii acceptă octetul) și MULTIPROC_DATA (bitul 9 = 0 — doar nodul adresat continuă recepția) este gestionată în software înainte de fiecare câmp transmis.

4.3. Interfața cu utilizatorul

UserIO() este non-blocantă, apelată la fiecare iterație a buclei principale. Preia intrări de la tastatură și terminal PC (UART0) și afișează simultan pe LCD și terminal. Flag-ul AFISARE coordonează partajarea afișajului între mesajele protocolului și textul introdus de utilizator.

5. Programul principal

5.1. Inițializarea resurselor

main() inițializează resursele în următoarea ordine: dezactivare WDT (implicit activ după resetare), configurare SYSClk pe oscilatorul extern de 25 MHz, inițializare UART1 pentru RS-485 (9 biți, 115200 baud) cu transceiver activ, configurare crossbar switch și porturi I/O, inițializare LCD și tastatură, inițializare UART0 pentru terminalul PC (8 biți, 115200 baud), inițializare Timer0 pentru generarea timeout-urilor în UART1_Getch_TMO(), activare globală întreruperi (EA = 1), golire buffere retea[i].full = 0 pentru toți cei 3 indecși, și afișare meniu inițial.

5.2. Buclea principală și automatul FSA_COM

Buclea while(1) alternează două task-uri independente: gestionarea comunicației prin switch(STARE_COM) și interfața cu utilizatorul prin UserIO(). Această structură non-blocantă asigură că interfața rămâne responsivă chiar când comunicația este în așteptare. Automatul FSA_COM implementează protocolul Master-Slave prin 4 stări (descrise în graful de tranziții de mai sus):

Starea S0 – Recepție / Așteptare (Slave):

Nodul apelează RxMesaj(ADR_NOD) cu timeout extins (2×WAIT + ADR_NOD×WAIT ms). Tranziții:

- TMO: nu a primit nimic → se autoprocldă master (TIP_NOD = MASTER, ADR_MASTER = ADR_NOD, i = 0, STARE_COM = 2);
- ROK: a primit date → Afișare_mesaj() → STARE_COM = 1;

- POK: a primit o interogare POLL → STARE_COM = 1 (pregătește răspuns).

Starea S1 – Răspuns (Slave):

Slave-ul caută în tabloul retea[] un buffer plin (reteza[i].full = 1). Dacă găsește: • setează adresa_hw_dest = ADDR_MASTER și apelează TxMesaj(i) pentru transmitere.

Dacă nu găsește nimic:

- construiește un POLL_MES implicit (adresa_hw_dest = ADDR_MASTER, tipmes = POLL_MES) și îl transmite.

Revine în STARE_COM = 0 după transmitere.

Starea S2 – Interogare (Master):

Selectează următorul slave: $i = (i+1) \% \text{NR_NODURI}$, sărind propria adresă. Setează retea[i].bufbin.adresa_hw_dest = i. Dacă retea[i].full = 1: apelează TxMesaj(i) cu datele. Dacă nu: construiește un POLL_MES și apelează TxMesaj(i). Tranzitează în STARE_COM = 3.

Starea S3 – Așteptare răspuns (Master):

Apelează RxMesaj(i) cu timeout de WAIT ms. Rezultate:

- TMO: slave-ul nu a răspuns → afișează numărul nodului în eroare;
- ROK: slave-ul a trimis date → Afisare_mesaj();
- POK: slave-ul a confirmat prezența fără date.

După procesarea răspunsului, masterul revine în STARE_COM = 2 pentru interogarea slave-ului următor.

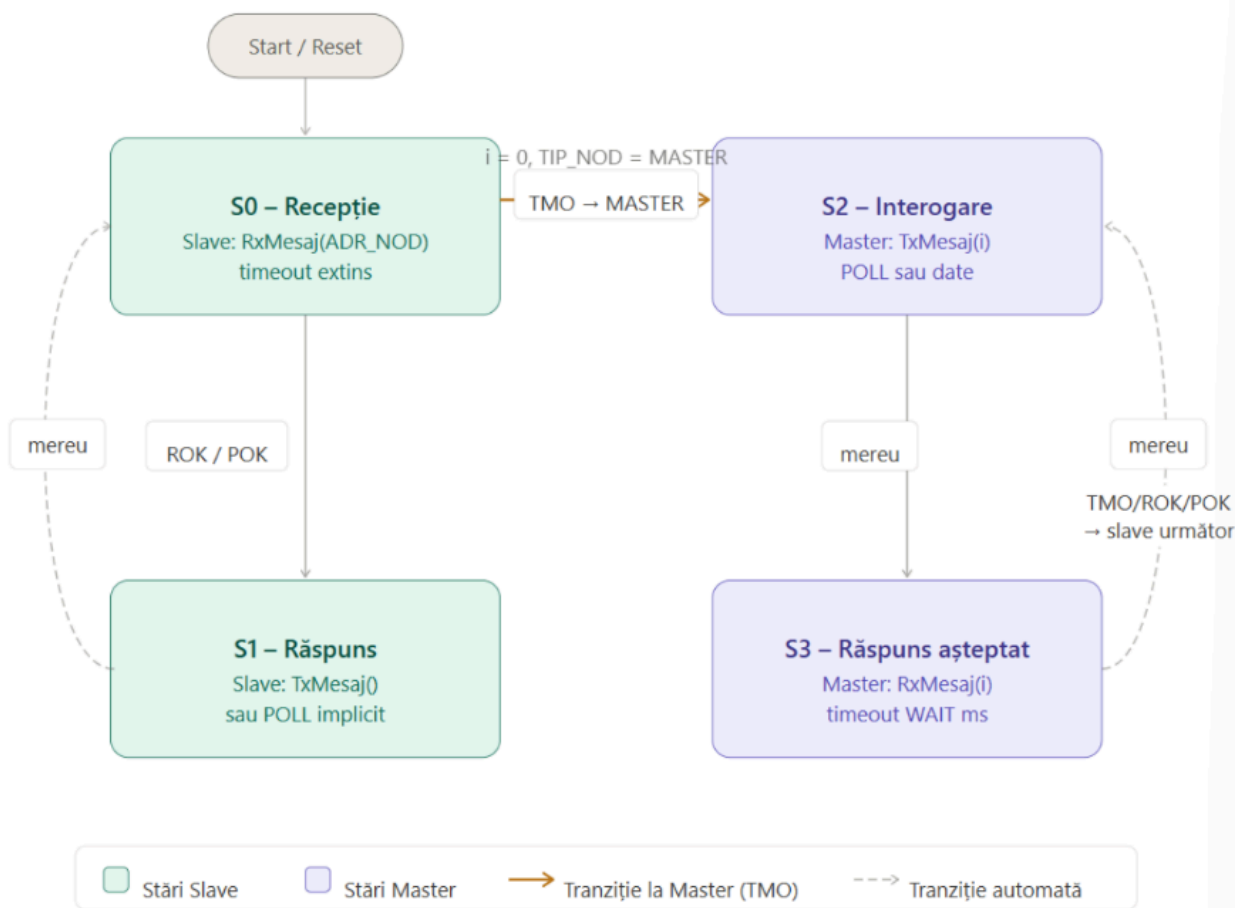
5.3. Graful de tranzitii FSA_COM

Stările verzi (S0, S1) sunt stările de Slave — nodul ascultă sau răspunde.

Stările violet (S2, S3) sunt stările de Master — nodul interoghează și așteaptă răspunsul.

Săgeata portocalie (S0 → S2) este tranziția specială: când slave-ul nu primește nimic în intervalul de timeout, se autoproclamă master și sare direct la interogare.

Săgețile punctate (arcurile laterale) sunt tranzițiile automate: S1 → S0 mereu după răspuns, S3 → S2 mereu după procesarea răspunsului de la slave.



6. Interfața cu utilizatorul

6.1. Automatul de stare FSA_IO (STARE_IO) din UserIO.c

Task-ul UserIO() implementează un automat de stare cu 3 stări (STARE_IO = 0, 1, 2), apelat la fiecare iterație a buclei principale. Funcția este non-blocantă: dacă nu există o tastă disponibilă, returnează imediat. Variabilele locale și de stare (tasta, dest, lng) sunt declarate static pentru a persista între apeluri.

S0 – IDLE (așteptare comandă):

Afișează meniul și așteaptă o comandă. AFISARE = 1. Tranziții la tastă: • '1'

→ inițiază transmisia unui mesaj: afișează 'Tx Msg:> Nod =' pe LCD și terminal, AFISARE = 0, STARE_IO = 1;

- '2' → afișează starea nodului curent (Master/Slave, adresă proprie), AFISARE = 0, STARE_IO = 1.

S1 – Selectare destinație:

Așteaptă o cifră între '0' și '2', diferită de ADR_NOD. Dacă buffer-ul destinației este deja plin (reteza[dest].full = 1):

- afisează eroarea "Buffer plin", STARE_IO = 0, Afisare_meniu().

Dacă buffer-ul este liber:

- completează câmpurile adresa_hw_dest (= dest pentru master, = ADR_MASTER pentru slave), adresa_hw_src = ADR_NOD, src = ADR_NOD, dest;
- cere introducerea textului mesajului: 'Mesaj: ' pe LCD și terminal, lng = 0, STARE_IO = 2.

Orice altă tastă: STARE_IO = 0, Afisare_meniu().

S2 – Colectare text mesaj:

Fiecare caracter tastat (diferit de CR, LF, '*' și cu lng < 16) este stocat în reteza[dest].bufbin.data[lng++]. La apăsarea Enter, '*' sau la atingerea limitei: • reteza[dest].bufbin.lng = lng, reteza[dest].bufbin.tipmes = USER_MES, reteza[dest].full = 1;

- STARE_IO = 0, Afisare_meniu().

6.2. Partajarea interfeței cu afișarea mesajelor protocolului

Flag-ul AFISARE (definit în UserIO.c, utilizat ca extern în modulele de comunicare) controlează dacă funcțiile din task-ul de comunicare pot scrie pe afișaj:

- AFISARE = 1 în starea S0 și la revenirea în S0 prin Afisare_meniu(): afișarea din protocol este permisă;
- AFISARE = 0 în stările S1 și S2: mesajele din protocol nu suprascriu textul introdus de utilizator.

Funcțiile Afisare_meniu() și Afisare_mesaj() scriu pe LCD (linia 0 pentru starea nodului, linia 1 pentru mesaje) și pe terminalul PC (UART0). Funcția Error() afișează mesajele de eroare în același mod.

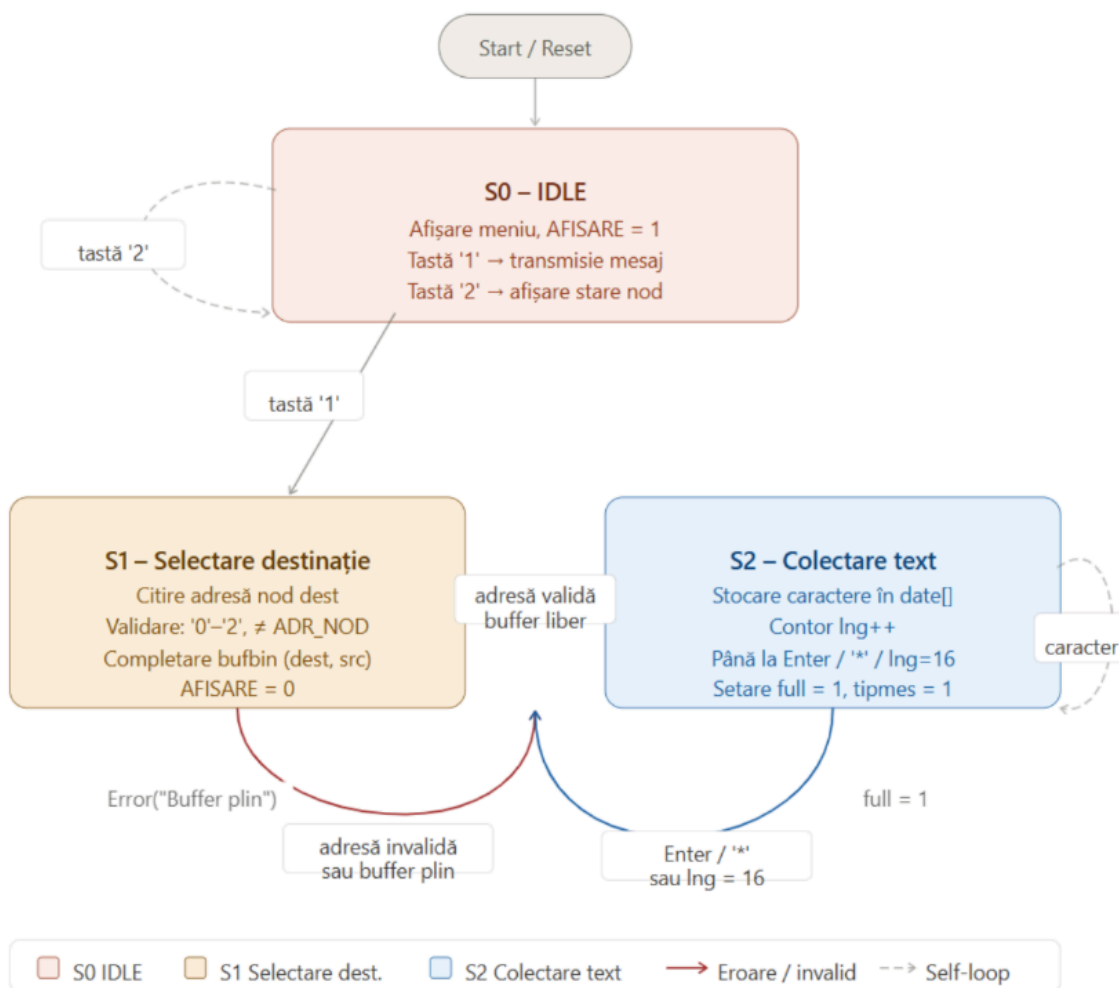
6.3. Graful de tranzitii FSA_IO

S0 roz — starea de așteptare (IDLE): meniul este afișat, AFISARE = 1. De aici pornesc toate tranzițiile.

S1 portocaliu — selectarea destinației: utilizatorul introduce adresa nodului destinație. Două ieșiri posibile: adresă validă și buffer liber → merge în S2; adresă invalidă sau buffer plin → eroare și întoarcere în S0.

S2 albastru — colectarea textului: caracterele sunt stocate în date[] unul câte unul (self-loop). La Enter / '*' / lng = 16 → full = 1, tipmes = USER_MES, și întoarcere în S0. **Self-loop-ul S0** (săgeată punctată stânga) — tasta '2' afișează starea nodului și rămâne în S0.

Săgețile roșii — tranzițiile de eroare (buffer plin, adresă invalidă) care revin în S0 cu mesaj de eroare.



7. Transmisia mesajelor

7.1. Funcția TxMesaj(i)

Transmite conținutul buffer-ului `reteza[i]` pe magistrala RS-485. Prima operație este calculul sumei de control LRC prin XOR cumulativ al câmpurilor mesajului: pentru `POLL_MES` doar `adresa_hw_dest` XOR `adresa_hw_src`, iar pentru `USER_MES` toate câmpurile inclusiv octeții din `date[]`. Rezultatul se stochează în `reteza[i].bufbin.sc` înainte de transmisie.

7.2. Secvența de transmisie

Funcția activează transceiver-ul RS-485 și comută UART1 în modul `MULTIPROC_ADRESA` (bitul 9 = 1), astfel că toți nodurile de pe magistrală recepționează primul octet. Se transmite `adresa_hw_dest` și se citește imediat eco-ul de pe magistrală: dacă octetul citit diferă de cel transmis sau apare timeout, a fost detectată o coliziune — transceiver-ul se dezactivează, se afișează mesajul de eroare, se aplică un delay de `WAIT` ms și funcția returnează fără a continua transmisia.

În absența coliziunii, UART1 comută în `MULTIPROC_DATA` (bitul 9 = 0) — nodurile cu altă adresă ignoră în hardware restul mesajului — și receptorul local se dezactivează pentru a elimina eco-ul. Se transmite secvențial `adresa_hw_src`, timpmes, iar dacă mesajul este de tip `USER_MES` se adaugă `src`, `dest`, `lng` și octeții din `date[]`. Ultimul câmp transmis este întotdeauna `sc`.

După transmiterea tuturor câmpurilor, receptorul se reactivează și se apelează `UART1_Getch(0)` pentru a bloca execuția până la ieșirea ultimului octet din registrul de transmisie. Flag-ul `full` este resetat la 0 doar dacă nodul este slave — masterul îl păstrează setat până primește confirmarea de la slave prin `RxMesaj`. În final, transceiver-ul RS-485 este dezactivat complet.

8. Recepția mesajelor

8.1. Descrierea funcției RxMesaj

Funcția `RxMesaj(i)` recepționează un mesaj de pe magistrala RS-485, cu comportament diferit în funcție de rolul curent al nodului (master sau slave). Parametrul `i` indică nodul de la care se așteaptă răspunsul (relevant pentru master).

Funcția returnează un cod de stare din mulțimea: `TMO` (timeout), `ROK` (mesaj de date corect), `POK` (poll/răspuns implicit corect), `ERA` (adresă hardware greșită), `TIP` (tip mesaj invalid > 1), `CAN` (mesaj incomplet/timeout la câmp), `OVR` (buffer destinație plin pe master), `ESC` (eroare sumă de control).

8.2. Faza de recepție adresă (mod multiprocesor)

Funcția configurează UART1 în mod MULTIPROC_ADRESA (recepționează doar octeții cu bitul 9 = 1), dezactivează emitătorul RS-485 și activează receptorul. Apoi:

- **Dacă nodul este MASTER:** apelează UART1_Getch_TMO(WAIT) cu timeout de 200 ms. Dacă timeout (variabila globală timeout = 1) → return TMO. Resetează `reteza[i].full = 0` (slave-ul a răspuns, mesajul anterior a ajuns). Dacă octetul primit != `ADR_NOD` (adresa proprie) → return ERA.
- **Dacă nodul este SLAVE:** execută o buclă do-while ce apelează UART1_Getch_TMO(2×WAIT + `ADR_NOD×WAIT`). Timeout → return TMO (magistrala liberă, se va prelua rolul de master). Bucla continuă până când octetul recepționat este egal cu `ADR_NOD` (mesaj adresat acestui nod).

8.3. Faza de recepție date

Odată recepționat octetul de adresă, funcția comută la MULTIPROC_DATA și inițializează variabila locală `screc = ADR_NOD`. Se recepționează secvențial fiecare câmp cu timeout individual de 5 ms (UART1_Getch_TMO(5)); la fiecare octet, valoarea este XOR-ată în `screc`:

- `adresa_hw_src` → dacă nod este SLAVE: `ADR_MASTER = adresa_hw_src` (actualizează adresa master-ului);
- `tipmes` → validat: dacă `tipmes > 1` → return TIP (valori valide: 0 = POLL_MES, 1 = USER_MES);
- Dacă `tipmes == USER_MES`: recepționează `src`, `dest`, `lng`;
- Dacă MASTER și `reteza[dest].full == 1`: return OVR (buffer destinație plin);
- Recepționează și stochează `date[0..lng-1]` cu XOR în `screc`;
- Dacă `tipmes == POLL_MES`: salvează `adresa_hw_src` și recepționează direct SC.

8.4. Verificarea sumei de control și stocarea mesajului

La finalul recepției, suma calculată (`screc`) este comparată cu octetul SC recepționat. Dacă `sc != screc` → return ESC (eroare detectată, mesajul este ignorat). Dacă `sc == screc`:

- **Pe MASTER (USER_MES):** stochează mesajul în `reteza[dest]` cu `adresa_hw_src = ADR_NOD` (pregătește retransmiterea), setează `reteza[dest].full = 1`, return ROK. Master-ul acționează ca router: la următoarea interogare a nodului dest, va transmite mesajul către slave-ul final.
- **Pe SLAVE (USER_MES):** stochează mesajul în `reteza[ADR_NOD]`, setează `reteza[ADR_NOD].full = 1`, return ROK.
- **POLL_MES (ambele roluri):** return POK – confirmare că nodul este activ, fără date de stocat.

9. Rezultate și concluzii

Proiectul a demonstrat implementarea completă a unui protocol de comunicație serială pe microcontrolere 8051, acoperând toate nivelurile: fizic (RS-485 cu adaptor TTL), legătură de date (format mesaj binar, LRC, modul multiprocesor UART) și aplicație (protocol Master-Slave cu detecție coliziuni, rutare slave-to-slave, interfata utilizator non-blocantă). Principalele aspecte tehnice confirmate:

- Modul multiprocesor al UART1 (9 biți) este esențial pentru eficiența protocolului: slave-urile ignoră în hardware octeții de date care nu le sunt adresați, reducând încărcarea procesorului;
- Detecția coliziunilor prin citirea eco-ului pe RS-485 funcționează fiabil la 115200 baud cu timeout de 2 ms;
- Automatul de stare non-blocant pentru interfata cu utilizatorul asigură că task-ul de comunicație nu este perturbat de operațiile de I/O;
- Arhitectura cu buffer per nod (tabloul retea[]) simplifică rutarea mesajelor și decuplează producția datelor de consumul lor;
- Decalajul de timeout pentru alegerea master-ului la pornire (proporțional cu ADR_NOD) previne eficient coliziunile inițiale.

Bibliografie

1. Silicon Laboratories, C8051F040/1/2/3/4/5/6/7 Mixed Signal ISP Flash MCU Family Data Sheet, Rev. 1.4, 2005.
2. MikroElektronika, BIG8051 Development System User Manual, Rev. 1.0, 2010.
12. Keil Elektronik GmbH / ARM, C51 Compiler User's Guide for Classic 8051 Devices, Version 9, 2012.
3. TIA/EIA-485-A: Electrical Characteristics of Generators and Receivers for Use in Balanced Digital Multipoint Systems, Telecommunications Industry Association, 1998.
4. Materialele didactice ale disciplinei Comunicații în sisteme de conducere – Microsistemul BIG8051, Microcontrolerul C8051F040, Mediul integrat Keil PK51 – Universitatea Tehnică "Gheorghe Asachi" din Iași, 2025.